

2.2 Theory Questions

Q1: Explain the pros and cons of using macros like:

```
#define U_prv(i) buffers[0][(i)+1]
#define U(i) buffers[1][(i)+1]
#define U_nxt(i) buffers[2][(i)+1]
```

The pros of using such macros is that you may have a smaller likelihood of doing off-by-one mistakes as it abstracts away buffer selection and accounting for the ghost boundaries.

Additionally, they decouple the buffer rotation from the call sites. When we move the sliding window forward it does not affect how we select the correct buffers using `U`, `U_prv`, and `U_nxt`.

The cons are that it may make it more difficult to actually take into account the ghost boundaries when you have to operate with them. E.g. doing `U(-1)` feels unnatural with the knowledge that `U` maps to a buffer which under normal circumstances can not be indexed by a negative integer.

Q2: Mention at least one other boundary condition that could have been applied instead of the Neumann (reflective) boundary condition. How will it affect the simulation?

Instead of using Neumann boundary condition, we could use *Dirichlet* boundary condition. This method sets the boundaries to some constant, often 0. This would be fine if the ends of the approximations never really change much, but for the wave function this would not work well as the endpoints move around a lot. A constant 0 at the start would create a sharp kink at the beginning of the simulation as the starting position for `U(0)` is not near 0. This error would then propagate through the wave as the iterations go.

Q3: What happens if you don't allocate memory in `T1`?

If memory allocation is not done, then the buffers would be just null pointers because of the way buffers is initialized:

```
real_t *buffers[3] = {NULL, NULL, NULL};
//                ^^^^^^^^^^^^^^^^^
//                all the buffers are null pointers.
```

`U(i)` would then become `buffers[1][(i) + 1]` and indexing into a null pointer (`buffers[1]`) is undefined behaviour.

Q4: What is the difference between:

```
float const *a;
```

and

```
float *const a;
```

`float const *a` is a float-pointer pointing to a constant float. In practice, this means we can modify what memory address we are pointing to, but we are not allowed to modify the value through the pointer.

`float *const a` is a constant pointer, pointing to a float. In practice, this means you can modify the float, but you can not point to a new float somewhere else in memory.